Queues

A `Queue` is an abstract data type that allows insertion at one end (the *tail*) and removal of items at the other end (the *head*). Queues store items in a **FIFO** (First In, First Out) fashion. For example, if you add the integers **1**, **2**, **3** to a queue of integers and then dequeue them you would get **1**, **2**, **3**. There are many useful applications of a queue such as the Windows Print Manager. Documents to be printed are queue to the Print Manager which prints them in the order they are received. Traditionally, the addition and removal operations are called **`enqueue`** (adding a new item) and **`dequeue`** (removing an item). Additionally an **`empty`** method is necessary to allow the user to check the queue condition. Dequeuing an empty queue creates an error condition.

The methods generally associated with queues are:

Method Summary	
`Object`	`dequeue()` Removes and returns the first element of the queue.
`boolean`	`empty()` Returns true if this queue is empty, otherwise returns false.
`void`	`enqueue(`*`Object`* **`o)`** Adds an element on the back of this queue.
`Object`	`peek()` Returns the first element of the queue without removing it. Returns null if the queue is empty

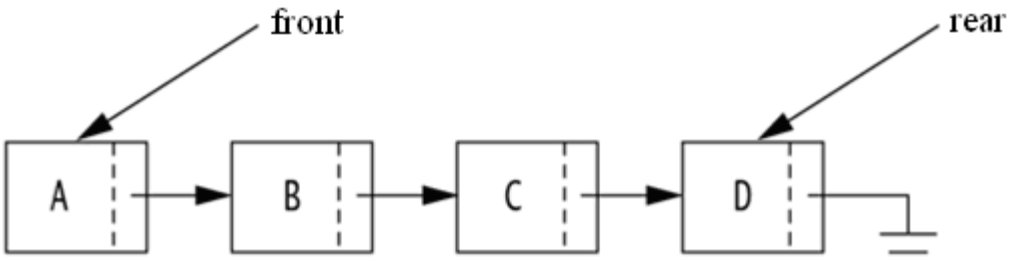
Queues can be implemented using either an array or a linked list data structure. The linked list implementation is more efficient because of the dynamic nature of a linked list.

Array Implementation

A queue can be implemented using an n element array called **`queue[0..n-1]`**. The rear of the queue corresponds to the last nonempty spot in the array, whose index could be remembered by an auxiliary variable called rear. Enqueuing an element on the queue simply involves placing that element in the array location queue[rear+1], and then incrementing rear by 1, first checking of course that rear does not exceed n-1. Dequeuing an element off the queue similarly involves returning the data in **`queue[0]`**, moving all the data elements forward one and decrementing rear. If rear equals 0 then the queue is empty and the **`dequeue`** method should return some sort of error. If one does not know the maximum number of elements to be held by the queue in advance one could alternatively implement the queue using a linked list with enqueue adding elements to the end of the list and **`dequeue`** removing the first element in the list.

Linked List Implementation

For implementations to be competitive with contiguous array implementations, we must be able to perform the basic linked list operations in constant time. Doing so is easy because the changes in the linked list are restricted to the elements at the two ends (front and rear) of the list.



Linked List implementation of a Queue class

To implement **`enqueue`**, we create a new node and attach it as the last element. We then advance the rear of the queue to the new node. An empty queue is represented by an empty linked list. Each operation is performed in constant time because, by restricting operations to the last (rear) node, we have made all calculations independent of the size of the list.

To implement **`dequeue`** we simply return the data stored in the first (front) node and advance front to the next node thereby removing the original front from the queue.

The queue itself is represented by a two data members, front, which is a reference to the first ListNode in the linked list and rear, which is a reference to the last **`ListNode`** in the linked list.

To implement a queue as a linked list you must define a **`ListNode`** class. The **`ListNode`** class can be nested in the **`Queue`** class or it can be a top-level class that is only package-visible, thus enabling the reuse of the class for other linked list implementations.

```
public class QueueNode<T>
{
    private T value;
    private QueueNode <T> next;

    public QueueNode () {
        value = null;
        next = null;
    }

    public QueueNode (T initValue, QueueNode <T> initNext) {
        value = initValue;
        next = initNext;
    }

    public T getValue() {
        return value;
    }

    public void setValue(T theNewValue) {
        value = theNewValue;
    }
```

```
    public ListNode<T> getNext() {
        return next;
    }

    public void setNext(ListNode<T> theNewNext) {
        next = theNewNext;
    }
}
```

The method **empty()** for a Linked List implementation of a queue could be written as follows:

```
public boolean empty() {
    return front == null;
}
```

The offer algorithm for a Linked List implementation:

- Instantiate a temporary **`QueueNode`** passing the constructor the new data Object.
- If the front node is null then assign both the front and rear node the location of the new node otherwise assign *rear.next* the location of the new node then make rear equal to the new node.

The remove algorithm for a Linked List implementation:

- Declare a temporary Object variable assigning it the value of the data at the queue front.
- Assign front the location of *front.next*.
- If *front* is null make *rear* null
- Return the temporary Object variable.

Java Queue Interface

While Java provides a Stack class it does not directly implement a **`Queue`** class. Instead it provides a **`Queue`** interface. The **`LinkedList`** class implements the **`Queue`** interface as well as the **`List`** interface. So, to instantiate a **`Queue`** in Java you would make the following declaration:

```
Queue <data type> queue = new LinkedList<data type>();
```

The methods associated with the Java Queue interface:

Method Summary

boolean	**add**(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.
E	**element**() Retrieves, but does not remove, the head of this queue.
boolean	**offer**(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions. (Same as enqueue)
E	**peek**() Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
E	**poll**() Retrieves and removes the head of this queue, or returns `null` if this queue is empty.
E	**remove**() Retrieves and removes the head of this queue. (Same as dequeue)

Notice the rather conspicuous absence of an **`empty`** method. Since Java did not provide an empty method we test for an empty queue by using **`peek()`**, i.e**. `if (peek() == null)`** then the **`queue`** is empty.

ASSIGNMENT - Lab 04

Lab 04A - 70 point version

Write a **`Queue`** class that implements a `Queue` using a linked list as the underlying data structure. Double click on *_docs.bat* to see the methods associated with this `Queue` implementation. Load, compile and execute *QueueTester.java* to test your **`Queue`** class.

Output:

Adding 'Red', 'Green', 'Blue', 'Purple', and 'Orange' to the queue.

Testing the "toString()" method:
[Red, Green, Blue, Purple, Orange]

Testing for an empty() queue
false

Peek the front of the queue!
Red

remove the front of the queue!
Red

remove the front of the queue!
Green

Adding 'Yellow' to the queue.
Adding 'Brown' to the queue.

Testing the "toString()" method:
[Blue, Purple, Orange, Yellow, Brown]

Removing all items from the queue!
Blue
Purple
Orange
Yellow
Brown

Testing for an empty() queue
true

remove from an empty queue
Exception in thread "main" java.util.NoSuchElementException
at queue.Queue.dequeue(Queue.java:35)
at QueueTester.main(QueueTester.java:48)

Lab 04B - 80 point version

Write an **`Queue`** class that implements a Queue using an array as the underlying data structure. Double click on *_docs.bat* to see the methods associated with this `Queue` implementation. Load, compile and execute *QueueTester.java* to test your **`Queue`** class.

Output:

Adding 'Red', 'Green', 'Blue', 'Purple', and 'Orange' to the queue.

Testing the "toString()" method:
[Red, Green, Blue, Purple, Orange]

Testing for a full queue
true

Testing for an empty() queue
false

Peek the front of the queue!
Red

remove the front of the queue!
Red

remove the front of the queue!
Green

Adding 'Yellow' to the queue.
Adding 'Brown' to the queue.

Testing the "toString()" method:
[Blue, Purple, Orange, Yellow, Brown]

Removing all items from the queue!
Blue
Purple
Orange
Yellow
Brown

Testing for a full queue
false

Testing for an empty() queue
true

```
remove from an empty queue
Exception in thread "main" java.util.NoSuchElementException
        at queue.Queue.remove(Queue.java:28)
at QueueTester.main(QueueTester.java:54)
```

Lab 04C - 90 point version

Write a **`Queue`** class that implements a `Queue` using a circular array as the underlying data structure. This implementation should include all the methods included in the array implementation of a Queue. Load, compile and execute *QueueTester.java* to test your **`Queue`** class.

Output:

Adding 'Red', 'Green', 'Blue', 'Purple', and 'Orange' to the queue.

Testing the "toString()" method:
[Red, Green, Blue, Purple, Orange]

Testing for a full queue
true

Testing for an empty() queue
false

Peek the front of the queue!
Red

remove the front of the queue!
Red

remove the front of the queue!
Green

Adding 'Yellow' to the queue.
Adding 'Brown' to the queue.

Testing the "toString()" method:
[Blue, Purple, Orange, Yellow, Brown]

Removing all items from the queue!
Blue
Purple
Orange
Yellow
Brown

Testing for a full queue
false

Testing for an empty() queue
true

remove from an empty queue
Exception in thread "main" java.util.NoSuchElementException
at queue.Queue.remove(Queue.java:33)
at QueueTester.main(QueueTester.java:54)

Lab 04D - 100 point version

Consider the following class declaration for representing cards to be used in a program that simulates a card game.

```
public class Card implements Comparable<Card>
{
   // postcondition: returns the value of this card
   public int value();

   // postcondition:    returns the value 0 if the argument Card is equal to this Card;
   //                   a value less than 0 if this Card is less than the Card argument;
   //                   and a value greater than 0 if this Card is greater than the Card
   //                   argument.
   public int compareTo(Card c);

   // postcondition:    returns true if this Card has the same rank as the specified
   //                   object obj.
   public boolean equals(Object obj);

   // Constructors and other methods not shown
}
```

The card game to be simulated is a two-player game called “Compete” that is played with a set of cards, each having a numeric value. The object of the game is to win all of the cards.

Each player has a pile of cards and both piles start with the same number of cards. Play consists of a sequence of “rounds”. Play continues until at the end of a round at least one player is out of cards. A discard pile is created and used during the rounds and is empty at the beginning of each round.

You will write code to move cards from one pile to another and to play one round. Each pile of cards will be represented by a queue.

A round is played by executing the following steps until the round ends.

1. If exactly one player’s pile is empty, the other player adds all the cards in the discard pile to the bottom of his or her pile without changing the order. The round ends.
2. If both players’ piles are empty at the same time, both players’ piles remain empty. The round ends.
3. Each player turns over the top card from his/her pile.
4. If the cards match in value, both cards are added to the discard pile in either order and play returns to step 1.
5. If the cards do not match in value, the player whose card has the greater value adds any cards in the discard pile to the bottom of his/her pile without changing the order of the cards in the discard pile. He/she then adds both cards just played to the bottom of his/her pile in either order. The round ends.

(a) Write the method **appendQueue**, which is described as follows. **appendQueue** should remove all the cards from the argument source and add them to the argument destination in the same order.

```
// postcondition: all cards have been removed from source
//                and added to destination in the same order
public void appendQueue(Queue<Card> destination, Queue<Card> source) {

}
```

(b) You will write free function **oneRound**. **oneRound** takes the two players' piles of cards as parameters and carries out one round of the game. For your convenience the steps for one round are repeated here.

1. If exactly one player's pile is empty, the other player adds all the cards in the discard pile to the bottom of his or her pile without changing the order. The round ends.
2. If both players' piles are empty at the same time, both players' piles remain empty. The round ends.
3. Each player turns over the top card from his/her pile.
4. If the cards match in value, both cards are added to the discard pile in either order and play returns to step 1.
5. If the cards do not match in value, the player whose card has the greater value adds any cards in the discard pile to the bottom of his/her pile without changing the order of the cards in the discard pile. He/she then adds both cards just played to the bottom of his/her pile in either order. The round ends.

In writing **oneRound**, you may call any of the methods in the Card class and **appendQueue** from part (a).

Complete the method **oneRound**.

```
// precondition:  pile1.length() > 0; pile2.length() > 0
// postcondition: pile1 and pile2 have been updated according to the
//                rules of the game
public void oneRound(Queue<Card> pile1, Queue<Card> pile2) {

}
```